INFORME DE LABORATORIO

ARQUITECTURA DE COMPUTADORES

LABORATORIO 2

Exploración de discos y sistemas de archivos FAT32 y NTFS

Nombre del estudiante 1

Nombre del estudiante 2

Nombre del estudiante 3



UNIVERSIDAD CATÓLICA DE ORIENTE

FACULTAD DE INGENIERÍAS

2026

Introducción

En este laboratorio se recorre a mano el contenido de dos discos: una imagen con formato FAT32 y una imagen NTFS creada para la práctica. Sin montar ninguno de los dos, se calculan posiciones, se leen bytes y se comprueba cada resultado con un editor hexadecimal. Al final, el mismo trabajo se automatiza con scripts en Python.

Objetivos

Objetivo general

Entender cómo se organizan los datos en un disco, ubicando archivos y carpetas dentro de imágenes FAT32 y NTFS.

Objetivos específicos

- Leer el Boot Sector de FAT32 y calcular dónde está el directorio raíz.
- Encontrar archivos de distintos tipos y dos carpetas en la imagen FAT32.
- Explicar por qué el directorio raíz de FAT32 puede ocupar más de un cluster.
- Crear una imagen NTFS y localizar en ella archivos y carpetas con ayuda de la MFT.
- Automatizar el análisis con scripts en Python.

Conceptos básicos

- **Sector y cluster.** El sector es la unidad mínima de lectura (512 bytes). El cluster es un grupo de sectores y es la unidad mínima que se asigna a un archivo.
- **FAT32.** Usa una tabla (la FAT) que indica, para cada cluster, cuál es el siguiente del mismo archivo. Es simple y la leen casi todos los equipos (Microsoft Corporation, 2000).
- **NTFS.** Guarda cada archivo y carpeta como un registro en la tabla maestra (MFT). Incluso sus estructuras internas son archivos (Carrier, 2005).

Tabla 1

Comparación entre FAT32 y NTFS

Característica	FAT32	NTFS
Estructura principal	Tabla FAT con punteros a clusters	MFT: un registro de 1 KB por archivo
Directorio raíz	Cluster indicado en el Boot Sector	Siempre el registro 5
Ubicación de los datos	Cadena de clusters en la FAT	Data runs, o dentro del propio registro
Nombres	Formato 8.3 y nombres largos (LFN)	Hasta 255 caracteres UTF-16
Archivo borrado	Primer byte de la entrada = 0xE5	Bandera "en uso" = 0
Permisos y journaling	No	Sí

Fórmulas de FAT32 usadas en todo el informe (SPC = sectores por cluster, SPF = sectores por FAT):

```
Sector del cluster c = Reservados + NFAT x SPF + (c - 2) x SPC
Sector de la raíz    = Reservados + NFAT x SPF + (ClusterRaíz - 2) x SPC
Cluster inicial      = (High << 16) | Low
```

Parte 1. Localización de archivos en FAT32 (60 %)

Se trabajó con la imagen Arqui22025.img (507 201 024 bytes, 990 627 sectores), una copia de una memoria microSD sin tabla de particiones: el Boot Sector está en el byte 0. Los valores se obtuvieron con el analizador en Python y se comprobaron con dd | od.

Boot Sector

Tabla 2

Valores del Boot Sector de la imagen FAT32

Campo	Offset	Valor
Bytes por sector	0x0B	512
Sectores por cluster	0x0D	1
Sectores reservados	0x0E	1160
Número de FAT	0x10	2
Sectores por FAT	0x24	7612
Cluster del directorio raíz	0x2C	2

Figura 1

Sector 0 de la imagen con los campos resaltados

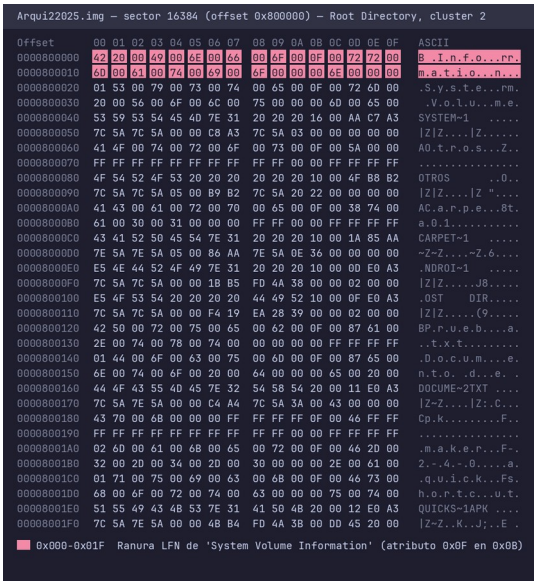


Ubicación del directorio raíz

- Inicio de la zona de datos: 1160 + 2 x 7612 = **16384**.
- Sector de la raíz: 16384 + (2 - 2) x 1 = **16384**, es decir el byte 8 388 608.

Figura 2

Inicio del directorio raíz (sector 16384)



Procedimiento general

1. Leer el Boot Sector y calcular dónde empieza la zona de datos.

2. Entrar al directorio (empezando por la raíz) y leer las entradas de 32 bytes. Si hay nombre largo, unir sus partes y comprobar el checksum.
3. Sacar el cluster inicial con (High << 16) | Low y el tamaño del archivo.
4. Convertir el cluster a sector con la fórmula del apartado de conceptos.
5. Seguir la cadena en la FAT hasta el final del archivo.
6. Comprobar los primeros bytes (la firma del tipo de archivo) con el editor hexadecimal.

Ejemplo: Documento de Prueba.txt

- Está en la raíz. Sus entradas de nombre largo se unen en "Documento de Prueba.txt"; el nombre corto es DOCUME~2.TXT. El checksum (0x87) coincide en las ranuras.
- Cluster inicial: High = 0 y Low = 0x003A, o sea **58**. Tamaño: **67 bytes**.
- Sector: 1160 + 2 x 7612 + (58 - 2) = **16440** (byte 8 417 280).
- FAT[58] marca fin de cadena, por lo que ocupa un solo cluster. El sector comienza con "Esto es", un texto.

Archivos encontrados

La tabla resume los archivos pedidos. Todas las firmas se verificaron en el primer sector de cada archivo.

Tabla 3

Archivos y carpetas localizados en la imagen FAT32

Categoría	Nombre	Cluster	Sector	Tamaño (B)
PDF	Organización y arquitectura de computadores.pdf	408694	425076	4 902 329
PDF	Montaje_y_mantenimiento_de_equipos.pdf	418269	434651	22 614 880
Aplicación	Mars4_5.jar	490097	506479	4 169 142
Aplicación	quickshortcutmaker-2-4-0.apk	59	16441	2 115 037
JPEG	Arquitecturas.jpg	485655	502037	355 589
JPEG	CPU.jpg	486350	502732	658 430
Word	Instructivo firma institucional.docx	7	16389	372 099
Word	Ordenadores De Placa Reducida.docx	405883	422265	1 439 229
Ensamblador	x2n.asm	498240	514622	419
Ensamblador	ejemplo-saltos.asm	490093	506475	426
Ensamblador	ejemploSuma.asm	490094	506476	92
Carpeta	Personal	462439	478821	0 (carpeta)
Carpeta	Carpeta01	341518	357900	0 (carpeta)

- La imagen no tiene archivos .exe; como aplicación se usó un paquete .apk (un archivo ZIP con firma PK).
- En total hay 4 PDF, 5 JPEG, 5 Word (.docx) y 7 archivos .asm. El Word del Instructivo está fragmentado (clusters 7 a 57 y 345745 a 346420), por eso hay que seguir la FAT.
- Personal contiene Fotos (con las imágenes y los .asm de Carpeta) y dos PDF; Carpeta01 contiene Docu y el PDF de Montaje.

Evidencia en el editor hexadecimal

Cada figura muestra la entrada del directorio (nombre largo, nombre 8.3, cluster y tamaño resaltados) y el primer sector de datos con su firma.

PDF

La primera figura muestra la firma %PDF-1.3 del primer PDF; la segunda, la entrada y la firma %PDF-1.4 del segundo.

Figura 3

Firma %PDF (25 50 44 46) en el sector 425076

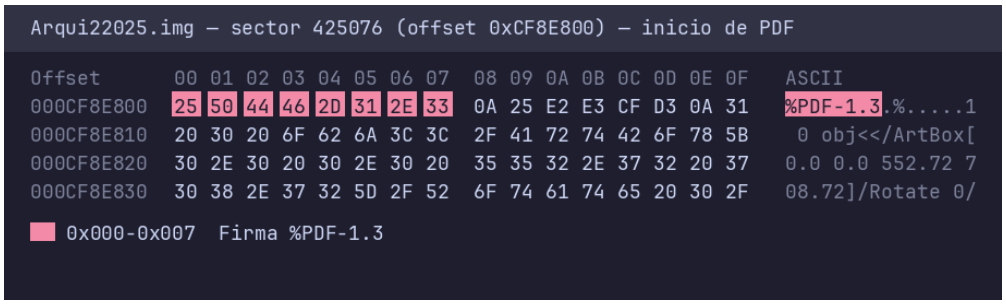
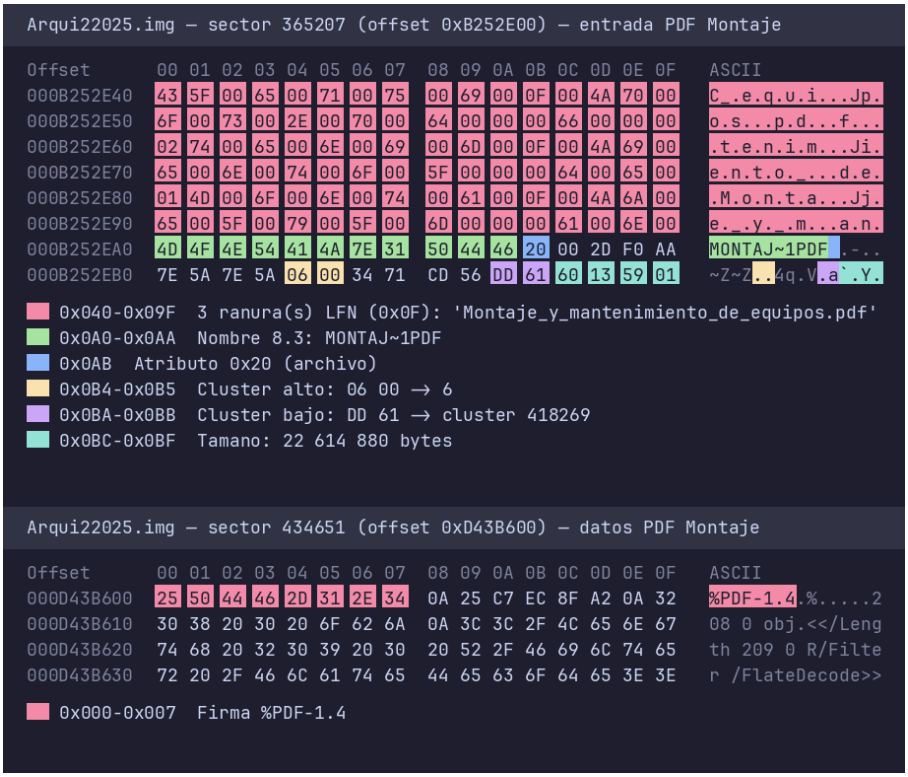


Figura 4

Entrada y datos de Montaje_y_mantenimiento_de_equipos.pdf

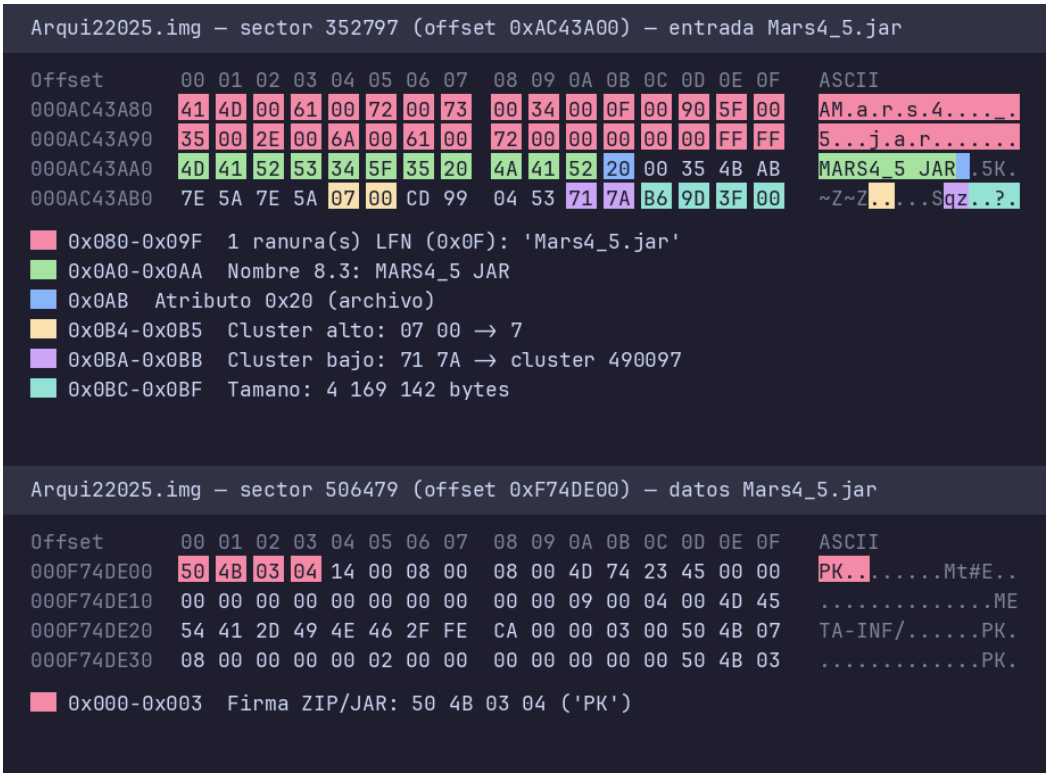


Aplicación

El archivo Mars4_5.jar (carpeta CarpeDocu) empieza con la firma ZIP 50 4B 03 04 en el sector 506479.

Figura 5

Entrada y datos de la aplicación Mars4_5.jar

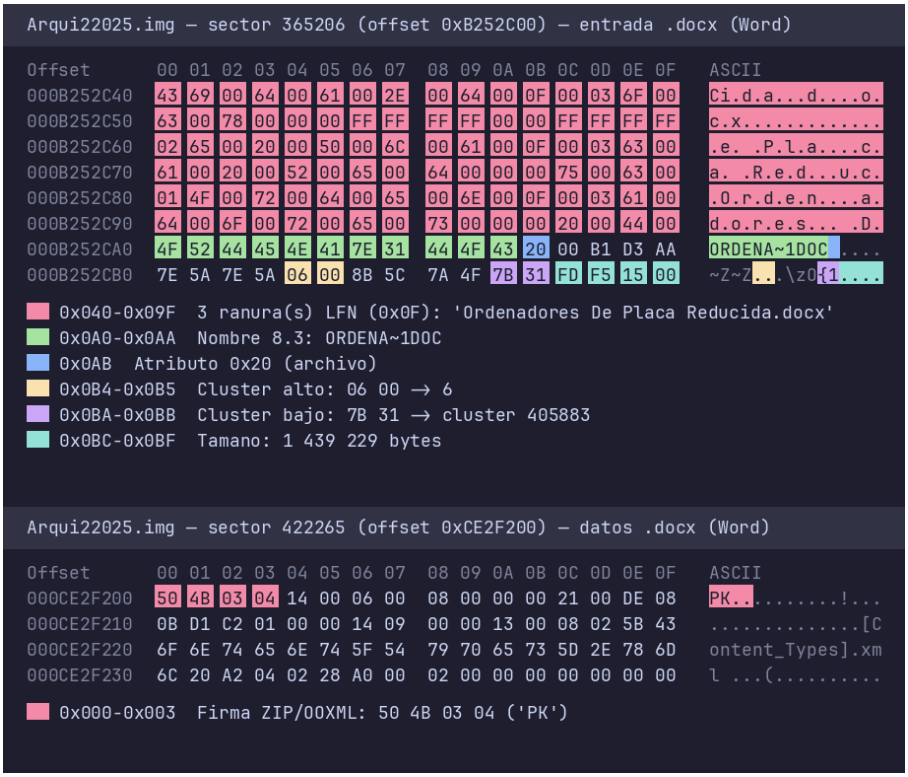


Word

El documento .docx también es un ZIP: su primer sector comienza con PK.

Figura 6

Entrada y datos del Word Ordenadores De Placa Reducida.docx



Ensamblador

Los .asm son texto plano: el contenido se lee en la columna ASCII.

Figura 7

Entrada y datos de x2n.asm

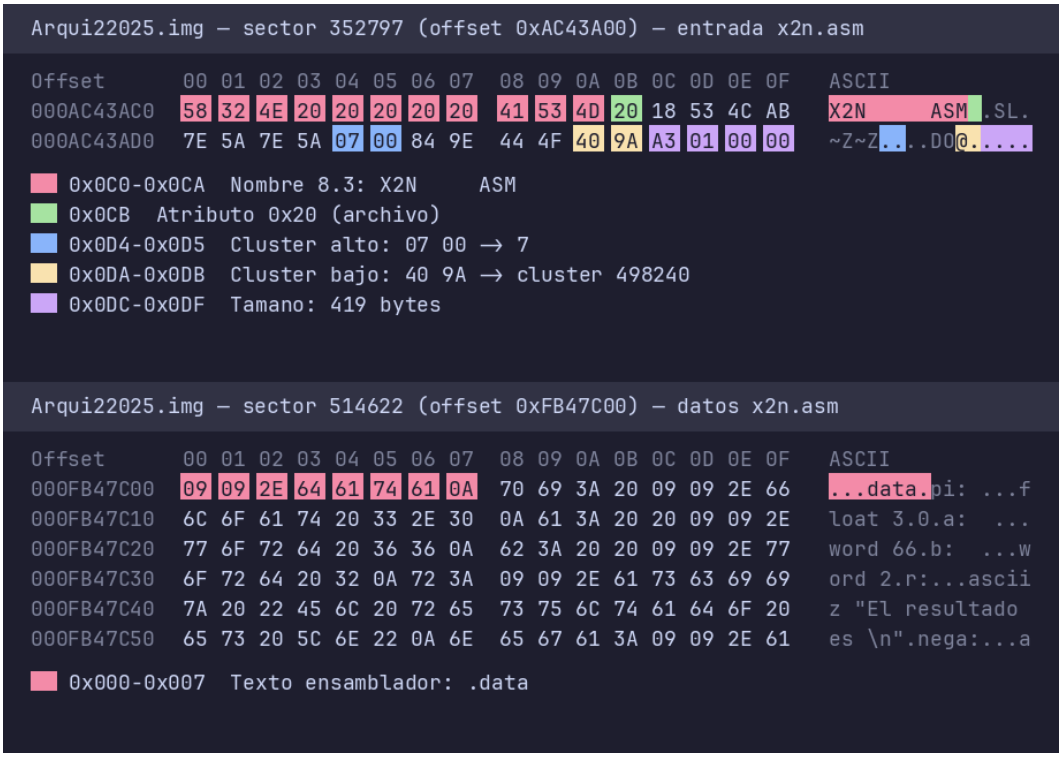
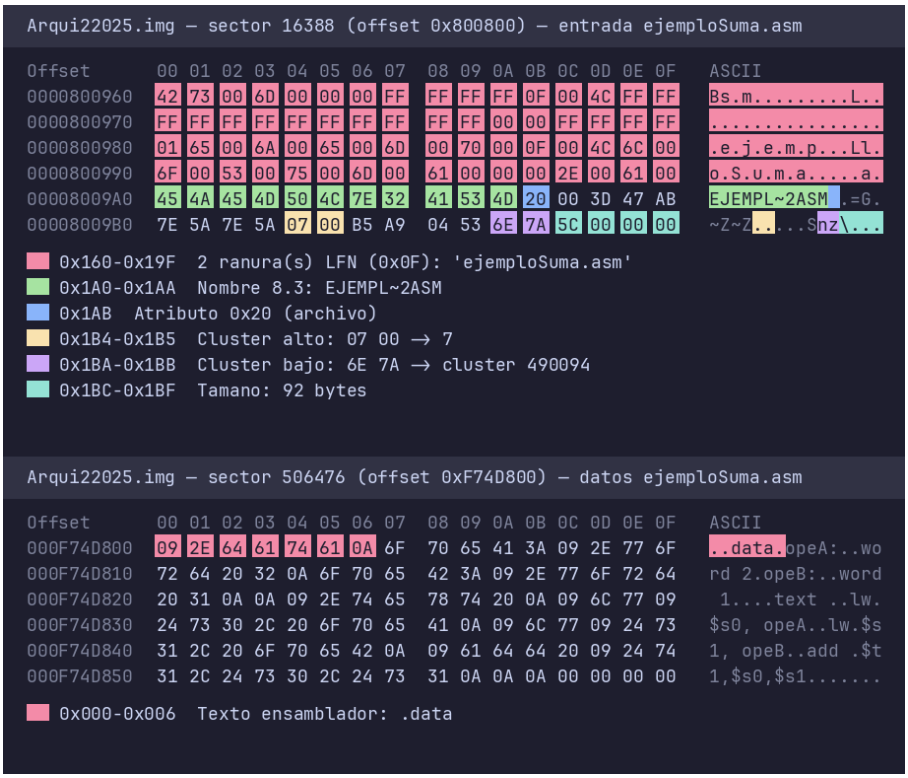


Figura 8

Entrada y datos de ejemploSuma.asm



JPEG

Ambas imágenes empiezan con FF D8 FF; la segunda es CPU.jpg.

Figura 9

Firma JPEG en el sector 502037 (Arquitecturas.jpg)

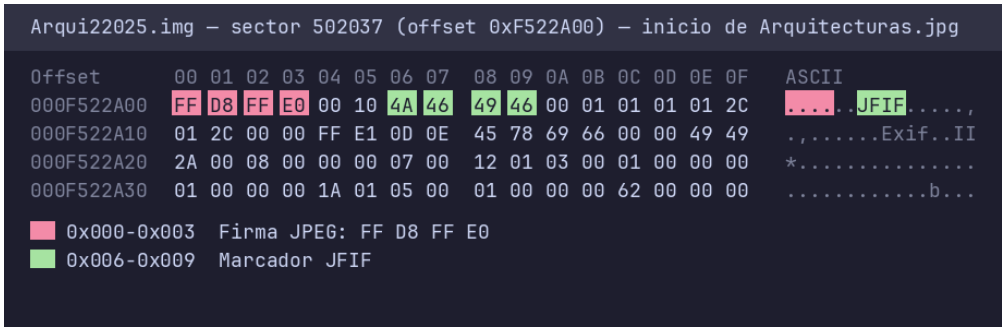
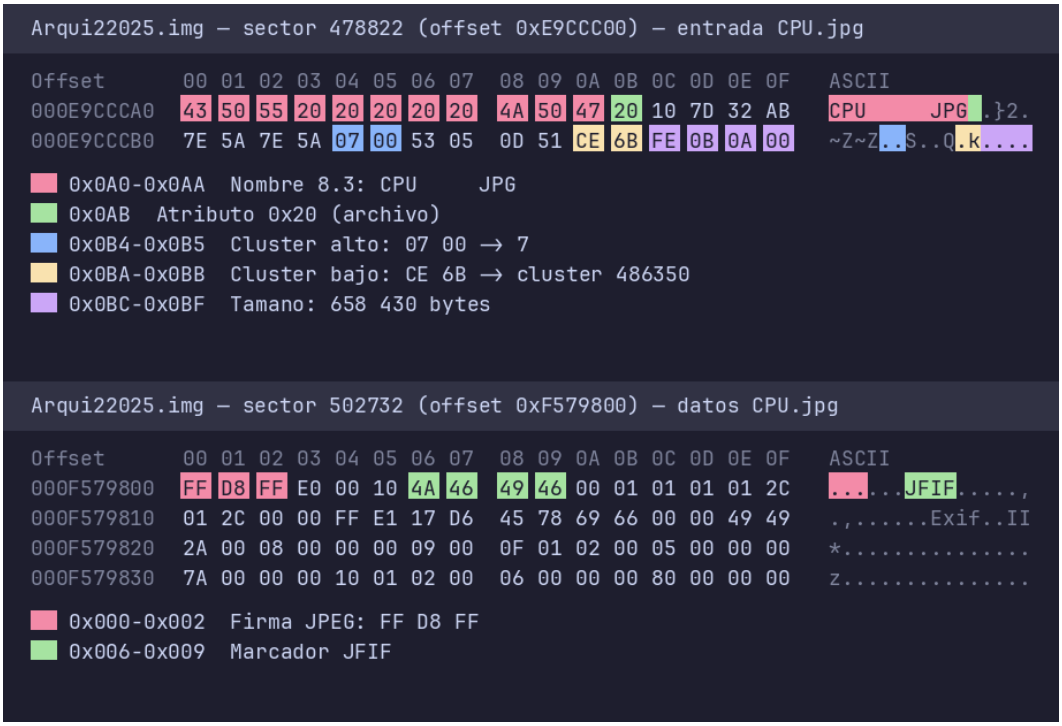


Figura 10

Entrada y datos de CPU.jpg



Carpetas

Un directorio es un cluster de entradas de 32 bytes que empieza con . (la propia carpeta) y .. (el padre).

Figura 11

Cluster de directorio de CarpeDocu

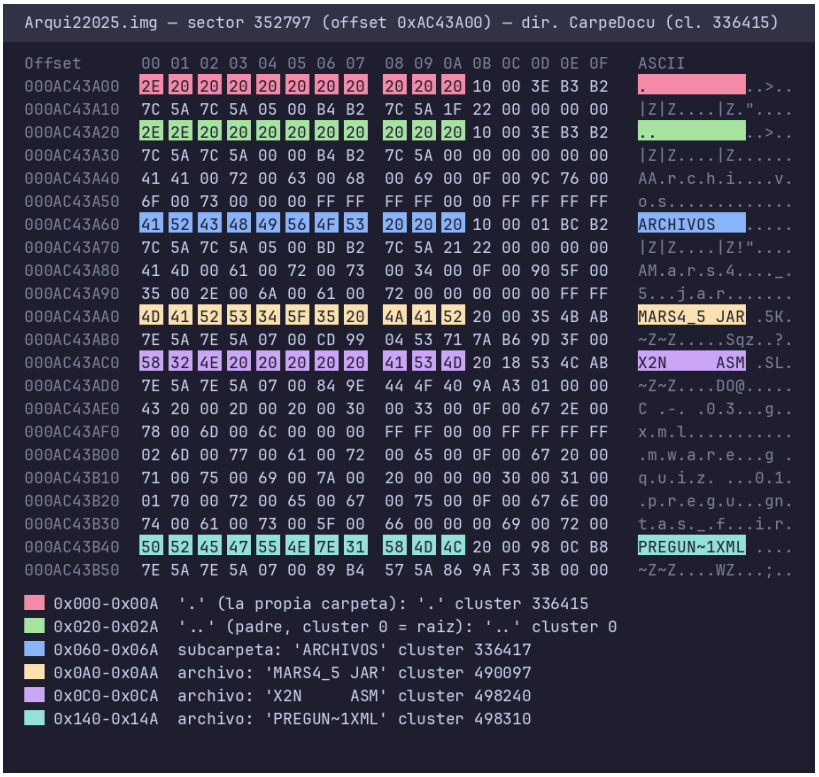
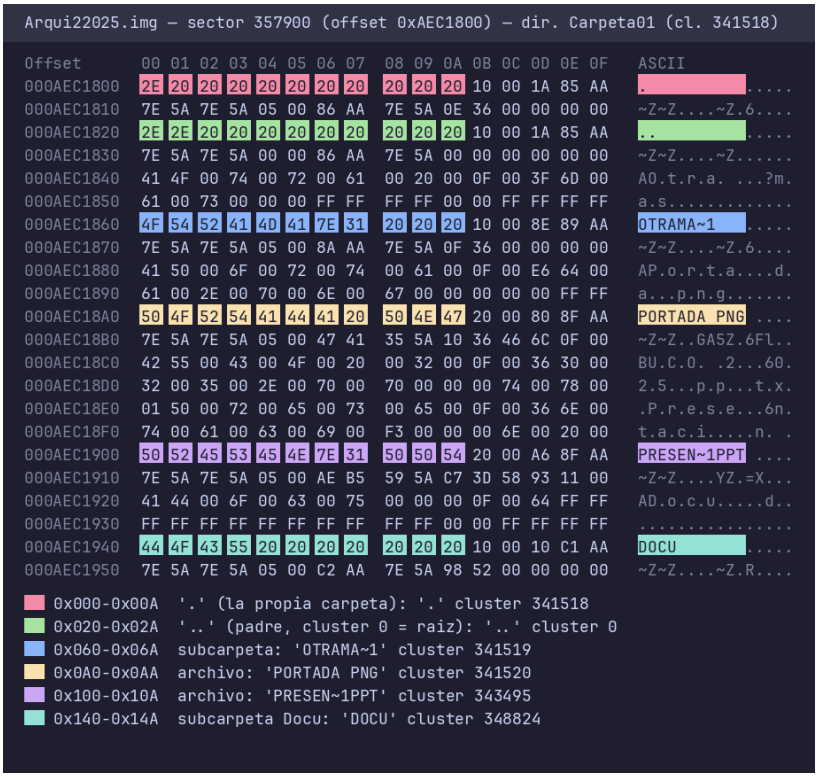


Figura 12

Cluster de directorio de Carpeta01



Parte 2. Problema del Root Directory (10 %)

Respuesta

- En esta imagen cada cluster mide 512 bytes y solo guarda **16 entradas de 32 bytes**. Los nombres largos usan varias entradas cada uno, así que el cluster se llena rápido.
- En FAT32 la raíz no es una zona fija: es un directorio normal que puede crecer enlazando más clusters en la FAT.
- Esos clusters pueden estar lejos y fuera de orden, por eso al leer solo el sector 16384 se ven apenas 5 entradas de archivo.
- **Solución:** leer la FAT del cluster actual para saber cuál es el siguiente y repetir hasta un valor de fin de cadena (0x0FFFFFF8 o mayor).

Procedimiento

1. Tomar el cluster raíz (2) y leer su sector (16384).
2. Leer FAT[2] en el byte 1160 x 512 + 2 x 4 = 593 928: el valor es 4190.
3. Repetir con los clusters siguientes hasta el fin de cadena.

4. Unir las entradas de todos los clusters.

Tabla 4

Cadena del directorio raíz

Cluster	Sector	Siguiente cluster (valor en la FAT)
2	16384	4190
4190	20572	6
6	16388	490095
490095	506477	Fin de cadena (0x0FFFFFFF)

La raíz tiene 14 entradas de archivo o carpeta vivas (5 carpetas y 9 archivos) y 2 borradas, repartidas en estos cuatro clusters.

Los enlaces se leen en la FAT: FAT[2] y FAT[6] en el sector 1160, FAT[4190] y FAT[490095] más adelante.

Figura 13

Sector 1160 (FAT 1) con FAT[2] y FAT[6]

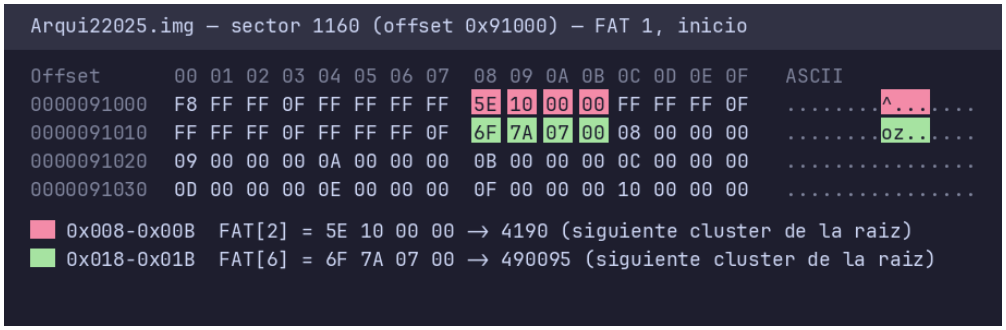
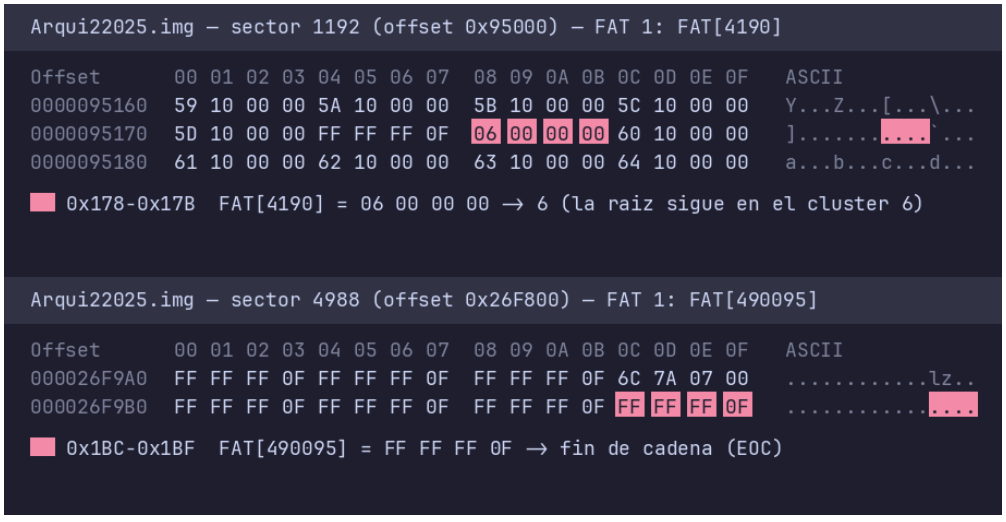


Figura 14

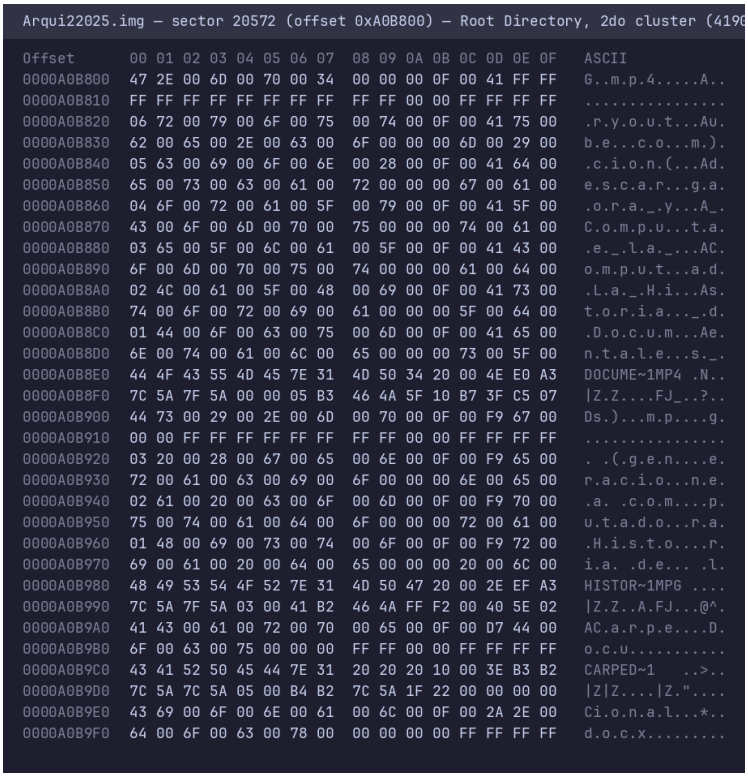
FAT[4190] = 6 y FAT[490095] = fin de cadena



Las entradas de la raíz están repartidas en los clusters 4190, 6 y 490095, además del cluster 2.

Figura 15

Segundo cluster de la raíz (cluster 4190, sector 20572)

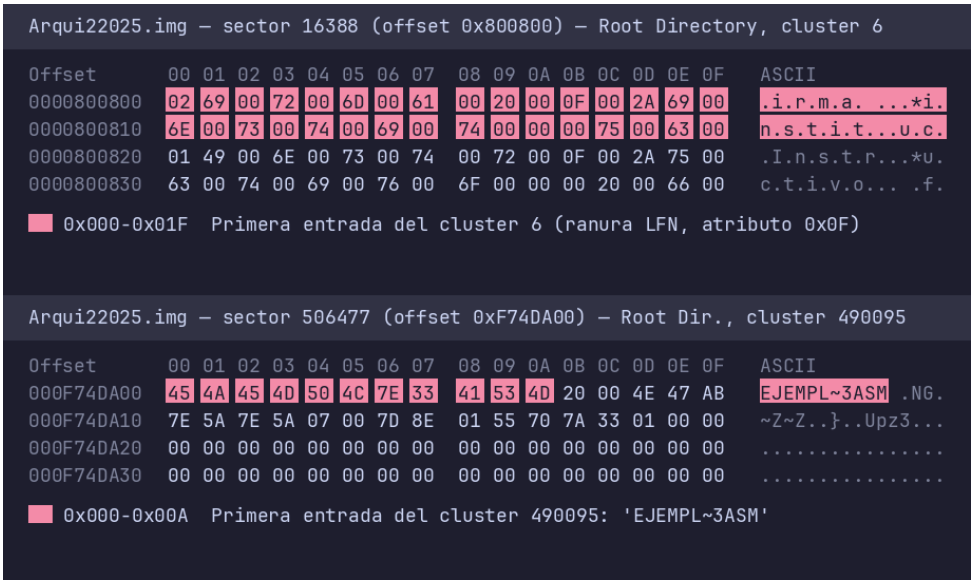


```
Arqui22025.img — sector 20572 (offset 0xA0B800) — Root Directory, 2do cluster (4190

Offset      00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0000A0B800  47 2E 00 6D 00 70 00 34  00 00 00 0F 00 41 FF FF   G..m.p.4.....A..
0000A0B810  FF FF FF FF FF FF FF FF  FF FF 00 00 FF FF FF FF   ................
0000A0B820  06 72 00 79 00 6F 00 75  00 74 00 0F 00 41 75 00   .r.y.o.u.t...Au.
0000A0B830  62 00 65 00 2E 00 63 00  6F 00 00 00 6D 00 29 00   b.e...c.o...m.).
0000A0B840  05 63 00 69 00 6F 00 6E  00 28 00 0F 00 41 64 00   .c.i.o.n.(...Ad.
0000A0B850  65 00 73 00 63 00 61 00  72 00 00 00 67 00 61 00   e.s.c.a.r...g.a.
0000A0B860  04 6F 00 72 00 61 00 5F  00 79 00 0F 00 41 5F 00   .o.r.a._.y...A_.
0000A0B870  43 00 6F 00 6D 00 70 00  75 00 00 00 74 00 61 00   C.o.m.p.u...t.a.
0000A0B880  03 65 00 5F 00 6C 00 61  00 5F 00 0F 00 41 43 00   .e._.l.a._...AC.
0000A0B890  6F 00 6D 00 70 00 75 00  74 00 00 00 61 00 64 00   o.m.p.u.t...a.d.
0000A0B8A0  02 4C 00 61 00 5F 00 48  00 69 00 0F 00 41 73 00   .L.a._.H.i...As.
0000A0B8B0  74 00 6F 00 72 00 69 00  61 00 00 00 5F 00 64 00   t.o.r.i.a..._.d.
0000A0B8C0  01 44 00 6F 00 63 00 75  00 6D 00 0F 00 41 65 00   .D.o.c.u.m...Ae.
0000A0B8D0  6E 00 74 00 61 00 6C 00  65 00 00 00 73 00 5F 00   n.t.a.l.e...s._.
0000A0B8E0  44 4F 43 55 4D 45 7E 31  4D 50 34 20 00 4E E0 A3   DOCUME~1MP4 .N..
0000A0B8F0  7C 5A 7F 5A 00 00 05 B3  46 4A 5F 10 B7 3F C5 07   |Z.Z....FJ_..?..
0000A0B900  44 73 00 29 00 2E 00 6D  00 70 00 0F 00 F9 67 00   Ds.)...m.p....g.
0000A0B910  00 00 FF FF FF FF FF FF  FF FF 00 00 FF FF FF FF   ................
0000A0B920  03 20 00 28 00 67 00 65  00 6E 00 0F 00 F9 65 00   . .(.g.e.n....e.
0000A0B930  72 00 61 00 63 00 69 00  6F 00 00 00 6E 00 65 00   r.a.c.i.o...n.e.
0000A0B940  02 61 00 20 00 63 00 6F  00 6D 00 0F 00 F9 70 00   .a. .c.o.m....p.
0000A0B950  75 00 74 00 61 00 64 00  6F 00 00 00 72 00 61 00   u.t.a.d.o...r.a.
0000A0B960  01 48 00 69 00 73 00 74  00 6F 00 0F 00 F9 72 00   .H.i.s.t.o....r.
0000A0B970  69 00 61 00 20 00 64 00  65 00 00 00 20 00 6C 00   i.a. .d.e... .l.
0000A0B980  48 49 53 54 4F 52 7E 31  4D 50 47 20 00 2E EF A3   HISTOR~1MPG ....
0000A0B990  7C 5A 7F 5A 03 00 41 B2  46 4A FF F2 00 40 5E 02   |Z.Z..A.FJ...@^.
0000A0B9A0  41 43 00 61 00 72 00 70  00 65 00 0F 00 D7 44 00   AC.a.r.p.e....D.
0000A0B9B0  6F 00 63 00 75 00 00 00  FF FF 00 00 FF FF FF FF   o.c.u...........
0000A0B9C0  43 41 52 50 45 44 7E 31  20 20 20 10 00 3E B3 B2   CARPED~1   ..>..
0000A0B9D0  7C 5A 7C 5A 05 00 B4 B2  7C 5A 1F 22 00 00 00 00   |Z|Z....|Z."....
0000A0B9E0  43 69 00 6F 00 6E 00 61  00 6C 00 0F 00 2A 2E 00   Ci.o.n.a.l...*..
0000A0B9F0  64 00 6F 00 63 00 78 00  00 00 00 00 FF FF FF FF   d.o.c.x.........
```

Figura 16

Clusters 6 y 490095 del directorio raíz



```
Arqui22025.img — sector 16388 (offset 0x800800) — Root Directory, cluster 6

Offset      00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0000800800  02 69 00 72 00 6D 00 61  00 20 00 0F 00 2A 69 00   .i.r.m.a. ...*i.
0000800810  6E 00 73 00 74 00 69 00  74 00 00 00 75 00 63 00   n.s.t.i.t...u.c.
0000800820  01 49 00 6E 00 73 00 74  00 72 00 0F 00 2A 75 00   .I.n.s.t.r...*u.
0000800830  63 00 74 00 69 00 76 00  6F 00 00 00 20 00 66 00   c.t.i.v.o... .f.

0x000-0x01F  Primera entrada del cluster 6 (ranura LFN, atributo 0x0F)

Arqui22025.img — sector 506477 (offset 0xF74DA00) — Root Dir., cluster 490095

Offset      00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
000F74DA00  45 4A 45 4D 50 4C 7E 33  41 53 4D 20 00 4E 47 AB   EJEMPL~3ASM .NG.
000F74DA10  7E 5A 7E 5A 07 00 7D 8E  01 55 70 7A 33 01 00 00   ~Z~Z..}..Upz3...
000F74DA20  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00   ................
000F74DA30  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00   ................

0x000-0x00A  Primera entrada del cluster 490095: 'EJEMPL~3ASM'
```

Parte 3. Localización de archivos en NTFS (20 %)

Creación de la imagen

Se creó lab_ntfs.img (200 MiB) con un script sin permisos de administrador: formatea con mkntfs (cluster de 4096 bytes, registro MFT de 1024), copia carpetas y archivos de prueba, y añade un MBR con la partición NTFS en el sector 2048. Antes de desmontar se borraron dos archivos y se añadió un flujo alternativo (ADS).

Procedimiento

1. Leer el MBR (sector 0): la partición NTFS empieza en el sector 2048.
2. Leer el Boot Sector (sector 2048): 8 sectores por cluster, MFT en el cluster 4, registros de 1024 bytes.
3. Calcular el sector de un registro: 2080 + 2 x N (el registro N de la MFT).
4. Leer el registro 5 (raíz) y buscar el nombre en su índice; si el directorio es grande, mirar también los bloques INDX.
5. Repetir por cada carpeta de la ruta; la entrada indica el número de registro del hijo.
6. En el registro del archivo mirar el atributo $DATA: si es residente, el contenido está dentro; si no, los data runs dan los clusters (sector = 2048 + LCN x 8).

Ejemplo: /Proyectos/2025/Lab2/informe/datos_largo.bin

Tabla 5

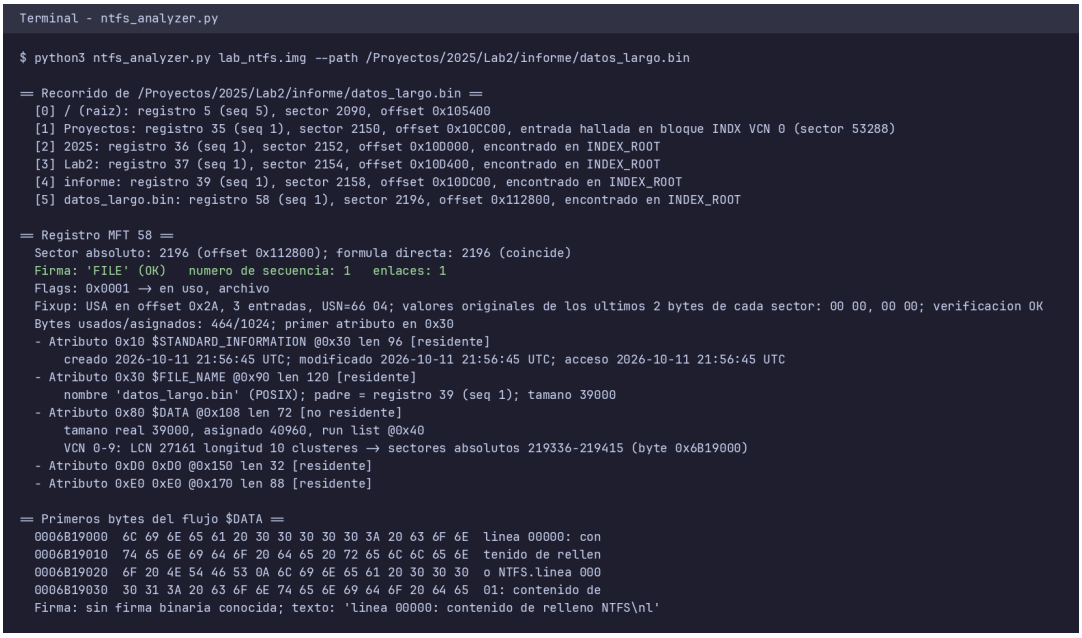
Recorrido de la ruta en la MFT

Elemento	Registro	Sector
Raíz	5	2090
Proyectos	35	2150
2025	36	2152
Lab2	37	2154
informe	39	2158
datos_largo.bin	58	2196

- El registro 58 tiene $DATA **no residente**: tamaño 39 000 bytes y un data run de 10 clusters que empieza en el LCN **27161**.
- Sector de los datos: 2048 + 27161 x 8 = **219336** (del 219336 al 219415).

Figura 17

Recorrido de la ruta con el analizador



Las figuras siguientes muestran el registro 0 con su data run, un bloque INDX y un registro borrado.

Figura 18

Registro 0 de la MFT: atributo $DATA y data run

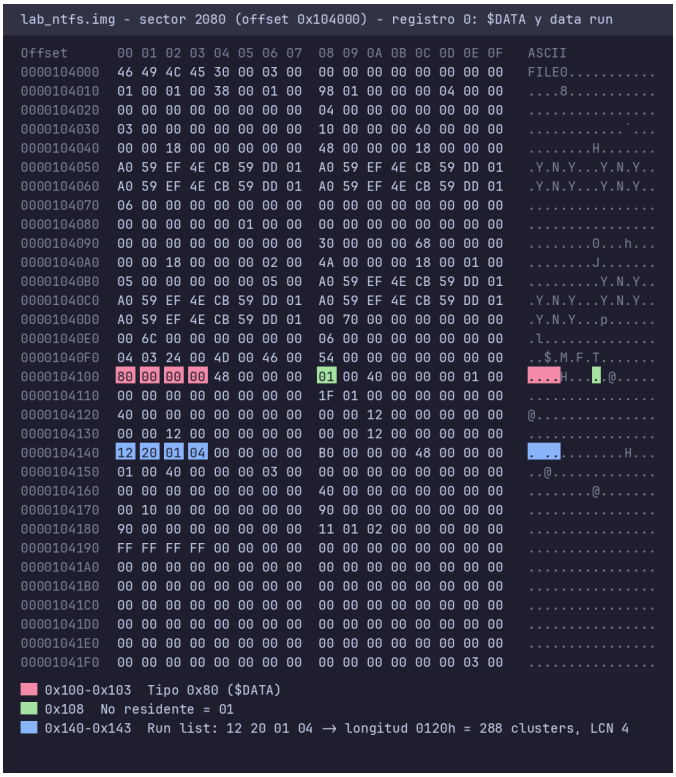


Figura 19

Bloque INDX del directorio /Muchos

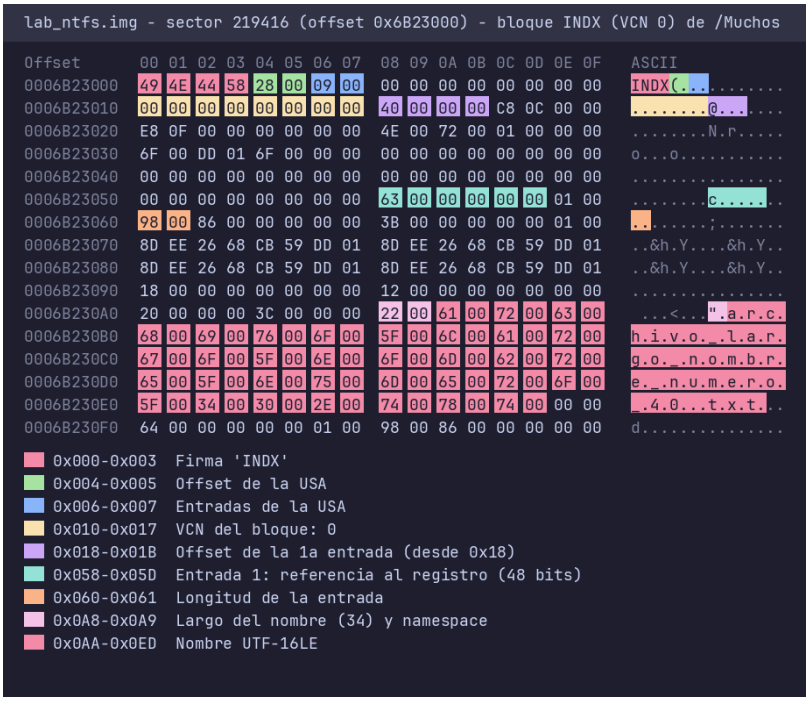
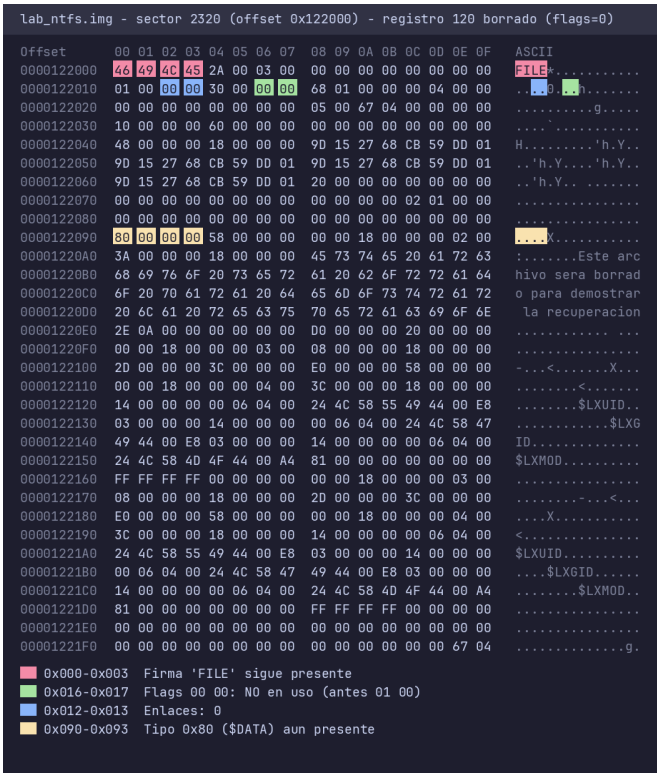


Figura 20

Registro 120 borrado (bandera en uso = 0)



Hallazgos

- **Residente y no residente:** pequeno.txt (60 bytes, registro 50) cabe dentro del registro; datos_largo.bin no, y usa clusters aparte.
- **Directorio grande:** /Muchos (registro 59, 60 archivos) no cabe en $INDEX_ROOT y usa $INDEX_ALLOCATION, con 5 bloques INDX (sector del primero: 219416).
- **Archivo borrado:** los registros 120 y 121 quedaron con la bandera "en uso" en 0; sus datos siguen en el disco y se recuperaron (el hash coincide con el original). El driver ntfs3 elimina el nombre, por eso Sleuth Kit los lista como OrphanFile-120 y -121.
- **Flujo alternativo (ADS):** nota.txt (registro 51) tiene un segundo $DATA llamado secreto que el Explorador no muestra.

Validación con The Sleuth Kit

Los comandos mmls, fsstat, fls e icat -o 2048 dieron los mismos números de registro y el mismo contenido (hash SHA-256) que el analizador, para las 80 entradas asignadas, incluido el ADS.

Parte 4. Código en Python (bono)

- fat32_analyzer.py lee la imagen FAT32 y muestra el Boot Sector, la cadena de la raíz, el árbol de carpetas, los archivos por extensión y los borrados.
- ntfs/ntfs_analyzer.py hace lo propio con NTFS: MBR, Boot Sector, registros MFT, índices, rutas, borrados y flujos alternativos.
- Ambos usan solo la biblioteca estándar y no montan la imagen.

  Ejemplos de uso:

```
python3 fat32_analyzer.py Arqui22025.img --find pdf jpg docx apk asm
python3 -I ntfs/ntfs_analyzer.py ntfs/lab_ntfs.img --path /Proyectos/2025/Lab2/informe/datos_largo.bin
```

Resultado de las pruebas automáticas: 35 pruebas en FAT32 y 18 en NTFS, todas correctas.

Conclusiones

- Con unos pocos datos del Boot Sector (sectores reservados, tamaño de la FAT y cluster raíz) se puede calcular dónde está cualquier archivo de FAT32.
- El directorio raíz de FAT32 es una cadena de clusters, no una zona fija. En la imagen fueron cuatro (2, 4190, 6 y 490095).
- Algunos archivos y carpetas están fragmentados, así que siempre hay que seguir la FAT.
- En NTFS la búsqueda pasa por la MFT: se llega al archivo por el índice de cada carpeta y se leen sus datos en el registro o en los data runs.
- Borrar un archivo no borra sus datos: en FAT32 solo se marca la entrada y en NTFS se baja la bandera de uso, por lo que se pudieron recuperar.
- Automatizar el análisis y compararlo con Sleuth Kit dio confianza en los resultados obtenidos a mano.

Referencias

Brown, R. (1994). Ralf Brown's interrupt list. http://www.ctyme.com/rbrown.htm

Carrier, B. (2005). File System Forensic Analysis. Addison-Wesley.

Glendinning, G. A. (2010). The art of assembly language (2nd ed.). No Starch Press.

Linux kernel documentation. (s. f.). NTFS3 filesystem driver.

https://docs.kernel.org/filesystems/ntfs3.html

Microsoft Corporation. (2000). Microsoft Extensible Firmware Initiative FAT32 File System Specification, Version 1.03. Microsoft.

Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Tanenbaum, A. S. y Bos, H. (2015). Modern operating systems (4th ed.). Pearson Education.

The Sleuth Kit. (s. f.). The Sleuth Kit documentation. https://www.sleuthkit.org/sleuthkit/docs.php

Declaración de uso de inteligencia artificial

Para este laboratorio utilizamos un asistente de inteligencia artificial generativa (Claude, de Anthropic) como herramienta de consulta y apoyo. Lo usamos para aclarar conceptos de FAT32 y NTFS, revisar cálculos, recibir sugerencias sobre cómo organizar el documento y repasar la redacción y el código. El análisis, las decisiones y la verificación de los resultados con el editor hexadecimal y las demás herramientas los realizamos y revisamos nosotros, y nos hacemos responsables del contenido de este informe.